Practice Lab
SET - 17

Name : Hitesh C
Reg : 192524005
Course: Compiler Design
Code : CSA 1401

1. Type Equivalence of Type Expressions

Aim:

To write a program to check whether two given type expressions T1 = array(10, int) and T2 = array(10, int) are Equivalent or Not Equivalent.

Theory:

Type equivalence is used by the semantic analyzer to verify whether two types are compatible. Under structural equivalence, two types are equivalent if and only if they have the same constructor, same dimensions, and identical basic element types.

Given Type Expressions:

```
T1 = array(10, int)
T2 = array(10, int)
-------------------------------------------------------
· Constructor check : array == array  -> Match (True)
· Dimension check   : 10 == 10        -> Match (True)
· Base type check   : int == int      -> Match (True)
```

Output : Equivalent

Algorithm:

1. Start the program.
2. Define the structure for type expressions containing constructor, size, and element type.
3. Initialize the type expressions T1 and T2.
4. Compare the type constructor of T1 with T2 (array == array).
5. If constructors match, compare array dimensions (10 == 10).
6. Check if the underlying element types are identical (int == int).
7. If all checks succeed, print "Equivalent"; otherwise print "Not Equivalent".
8. Stop.

Result:

The given type expressions T1 and T2 were evaluated for structural equivalence and displayed as Equivalent.

2. Left Factoring on the Given Grammar

Aim:

To write a C program to perform left factoring on the grammar: S -> abc | abd, A -> a, and display the resulting left-factored grammar.

Theory:

Left factoring is a grammar transformation technique used in top-down / predictive parsers to eliminate common prefixes from alternative productions of a non-terminal. This avoids backtracking during parsing.

General Form:

A -> alpha beta1 | alpha beta2

Left Factored Form:

A -> alpha A'
A' -> beta1 | beta2

Given Grammar Rules:

S -> abc | abd
A -> a

- For S: Longest common prefix alpha = "ab", suffixes beta1 = "c", beta2 = "d"
- For A: No common prefix exists.

Resulting Grammar: S -> abS', S' -> c | d, A -> a

Algorithm:

1. Start the C program.
2. Read the non-terminals and their corresponding productions.
3. For production S -> abc | abd, identify the longest common prefix alpha = "ab".
4. Create a new non-terminal symbol S'.
5. Rewrite the production as S -> abS'.
6. Add the new production rule for suffixes: S' -> c | d.
7. For productions without common prefix (A -> a), retain them as-is.
8. Display the modified, left-factored grammar rules.
9. Stop.

Result:

The given grammar was successfully left factored into S -> abS', S' -> c | d, and A -> a and displayed.

3. LEX Program to Count Macros and Header Files

Aim:

To write a LEX program to count the number of Macros defined and Header files included in a C program (sample.c).

Theory:

LEX uses pattern matching via regular expressions. In C programs, header files are included using '#include' and macros are defined using '#define'. We define regular expressions to identify each directive and increment separate counters.

Input Source Program (sample.c):

```
#define PI 3.14
#include<stdio.h>
#include<conio.h>
void main()
{
    int a,b,c = 30;
    printf("hello");
}
```

Count Analysis:

- Macro detected:
  `#define PI 3.14`
  => Total Macros = 1
- Header files detected:
  `#include<stdio.h>, <conio.h>`
  => Total Headers = 2

Algorithm:

1. Start the LEX program.
2. Initialize counters: macro_count = 0 and header_count = 0.
3. In rules section, define regex for header files: ^#include[ ]*<[^>]+> { header_count++; }
4. Define regex for macros: ^#define[ ]+[a-zA-Z_][a-zA-Z0-9_]* { macro_count++; }
5. Add rule to ignore all other source characters: . or \n
6. In main(), open "sample.c" and assign it to file pointer yyin.
7. Call yylex() to scan the entire input file.
8. Print the final counts: Number of Macros and Header files.
9. Stop.

Result:

The LEX program successfully scanned sample.c and displayed: Number of Macros = 1, Number of Header files = 2.

4. LEX Program to Identify and Count Palindrome Words

Aim:

To write a LEX program to identify and count palindrome words from a given input text.

Theory:

A palindrome word reads identically forwards and backwards (e.g. madam, racecar, level). In LEX, tokens are matched using the regular expression [a-zA-Z]+. For each word token yytext, we check if yytext[i] == yytext[len - 1 - i]. If it matches completely, the word is a palindrome.

Example Identification:

Input: "madam speaks Malayalam at radar level"

1. "madam" -> Forward == Backward -> Palindrome word
2. "radar" -> Forward == Backward -> Palindrome word
3. "level" -> Forward == Backward -> Palindrome word

Palindromes found: madam, radar, level => Total Count = 3

Algorithm:

1. Start the LEX program.
2. Initialize integer counter: pal_count = 0.
3. In rules section, define pattern for word tokens: [a-zA-Z]+.
4. Inside the action rule for each matched word token:
   a. Compute length of word len = strlen(yytext).
   b. Check if yytext[i] equals yytext[len - 1 - i] for all 0 <= i < len/2.
   c. If equal, print the palindrome word and increment pal_count++.
5. Ignore whitespace, digits, and other characters using space, tab, and punctuation.
6. In main(), call yylex() to scan input until EOF.
7. Display the total number of palindrome words identified.
8. Stop.

Result:

All palindrome words were identified from the input and the total count was displayed successfully.